In [2]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# 2. Load the Kaggle dataset
df = pd.read_csv("Mall_Customers.csv")

# 3. Explore the dataset
print("First five records:")
print(df.head())

print("\nLast five records:")
print(df.tail())

print("\nDataset shape:")
print(df.shape)

print("\nData types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isnull().sum())

print("\nNumber of duplicate records:")
print(df.duplicated().sum())

print("\nSummary statistics:")
print(df.describe())


# 4. Select features and standardize them
features = df[["Annual Income (k$)", "Spending Score (1-100)"]]

scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)


# 5(a). Scatter plot
plt.figure(figsize=(8, 5))
plt.scatter(features["Annual Income (k$)"],
            features["Spending Score (1-100)"])

plt.title("Annual Income vs Spending Score")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.grid(True)
plt.show()


# 5(b). Pair plot
sns.pairplot(features)
plt.show()


# 6. Elbow Method
```

```
wcss = []

for k in range(1, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(scaled_features)
    wcss.append(model.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker="o")
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.xticks(range(1, 11))
plt.grid(True)
plt.show()


# 7. Build the K-Means model
# K = 5 is commonly used for this dataset.
# Check the elbow plot to confirm your choice.

k = 5

kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(scaled_features)

print("\nDataset with cluster labels:")
print(df.head())

# Save the clustered dataset
df.to_csv("Mall_Customers_Clustered.csv", index=False)
print("\nClustered dataset saved successfully!")


# 8. Visualize clusters and centroids
# Convert standardized centroids to original units
centroids = scaler.inverse_transform(kmeans.cluster_centers_)

plt.figure(figsize=(10, 6))

for i in range(k):
    cluster_data = df[df["Cluster"] == i]

    plt.scatter(
        cluster_data["Annual Income (k$)"],
        cluster_data["Spending Score (1-100)"],
        label=f"Cluster {i}"
    )

plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker="X",
    s=250,
    color="black",
    label="Centroids"
)

plt.title("Customer Segmentation using K-Means")
plt.xlabel("Annual Income (k$)")
```

```
plt.ylabel("Spending Score (1-100)")
plt.legend()
plt.grid(True)
plt.show()

print("\nCentroid coordinates:")
for i, centroid in enumerate(centroids):
    print(
        f"Cluster {i}: Income = {centroid[0]:.2f} k$, "
        f"Spending Score = {centroid[1]:.2f}"
    )


# 9. Number of customers in each cluster
print("\nCustomers in each cluster:")
print(df["Cluster"].value_counts().sort_index())


# 10. Average income and spending score for each cluster
cluster_analysis = df.groupby("Cluster").agg(
    Customer_Count=("CustomerID", "count"),
    Average_Income=("Annual Income (k$)", "mean"),
    Average_Spending_Score=("Spending Score (1-100)", "mean")
).round(2)

print("\nCluster Analysis:")
print(cluster_analysis)
```

First five records:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Last five records:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

Dataset shape:

(200, 5)

Data types:

CustomerID	int64
Gender	str
Age	int64
Annual Income (k$)	int64
Spending Score (1-100)	int64

dtype: object

Missing values:

CustomerID	0
Gender	0
Age	0
Annual Income (k$)	0
Spending Score (1-100)	0

dtype: int64

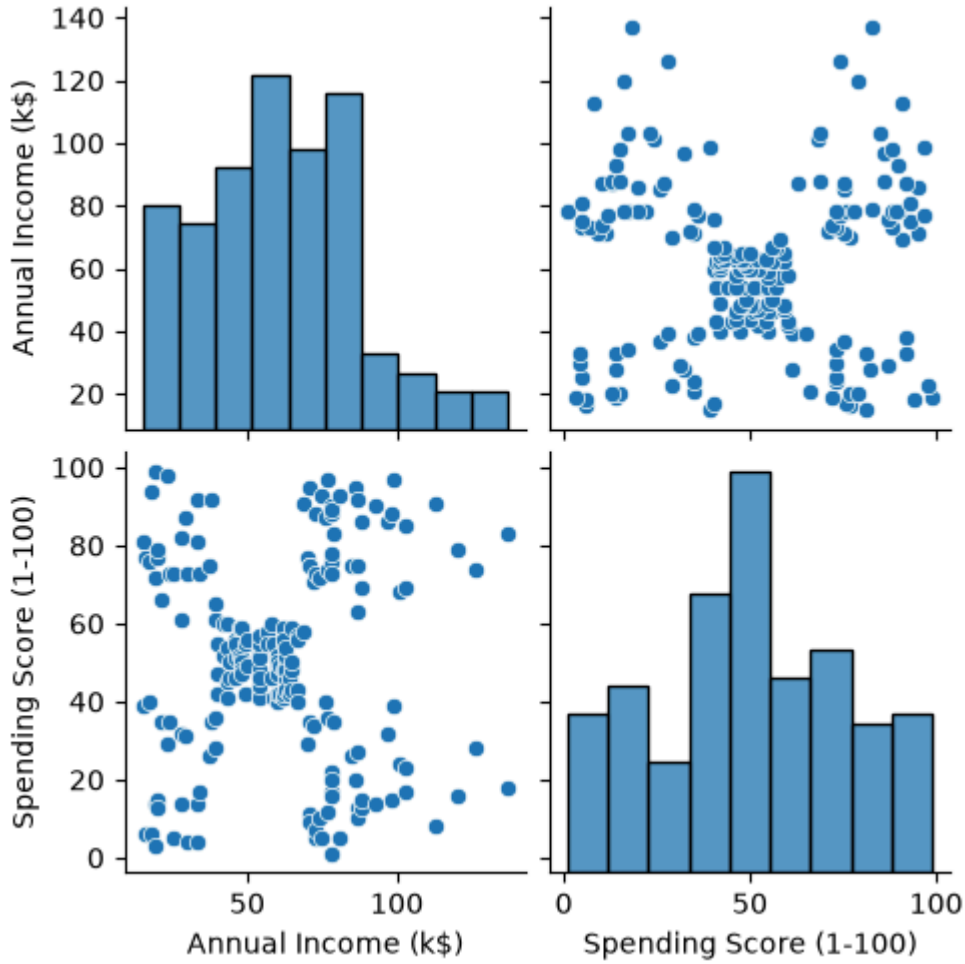
Number of duplicate records:

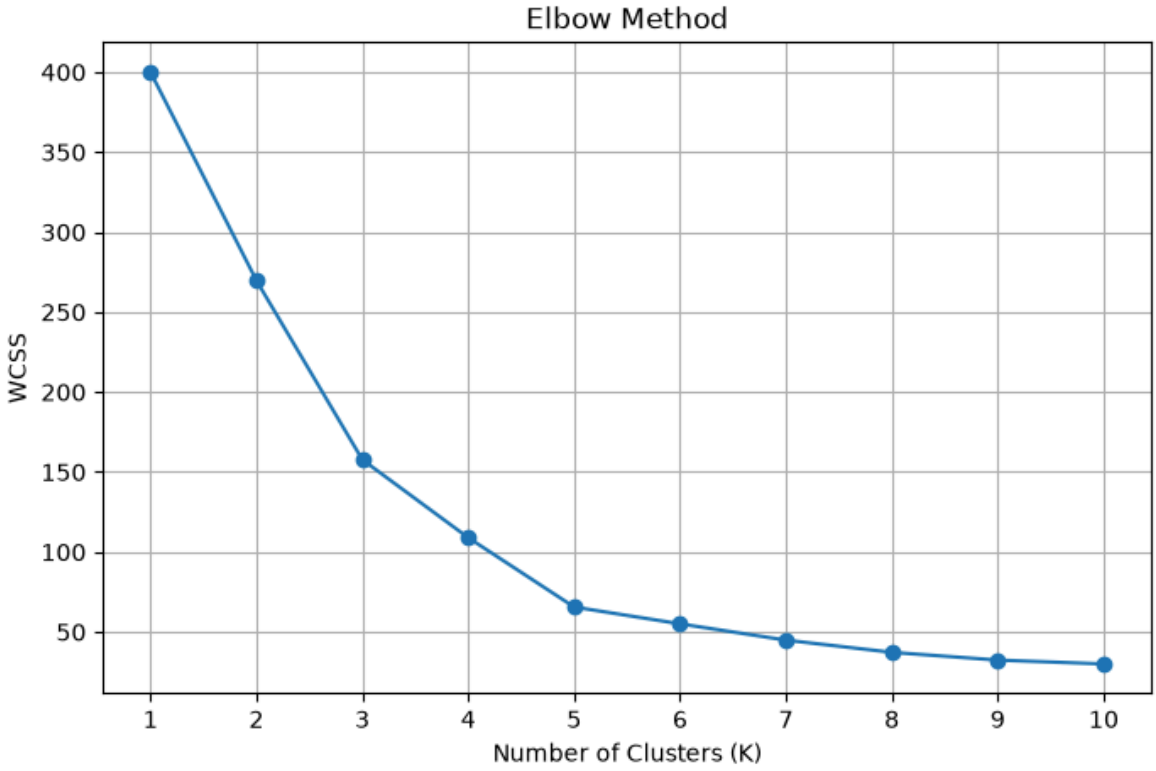
0

Summary statistics:

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000


Annual Income vs Spending Score
Spending Score (1-100)
Annual Income (k$)
0
20
40
60
80
100
20
40
60
80
100
120
140

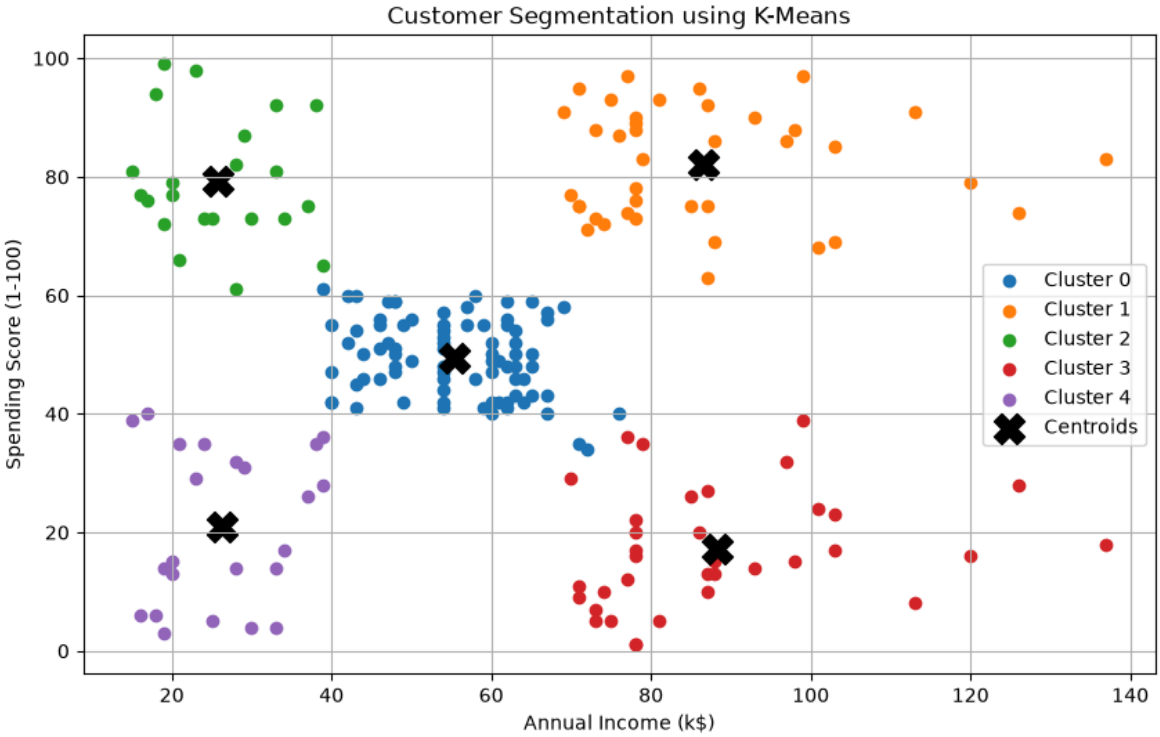

Annual Income (k$)
Spending Score (1-100)
Annual Income (k$)
Spending Score (1-100)
20
40
60
80
100
120
140
0
20
40
60
80
100
50
100
0
50
100




```
C:\Users\rena\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWa
rning: KMeans is known to have a memory leak on Windows with MKL, when there are
less chunks than available threads. You can avoid it by setting the environment v
ariable OMP_NUM_THREADS=1.
  warnings.warn(
Dataset with cluster labels:
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)  \
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

   Cluster
0        4
1        2
2        4
3        2
4        4

Clustered dataset saved successfully!
```




```
Centroid coordinates:
Cluster 0: Income = 55.30 k$, Spending Score = 49.52
Cluster 1: Income = 86.54 k$, Spending Score = 82.13
Cluster 2: Income = 25.73 k$, Spending Score = 79.36
Cluster 3: Income = 88.20 k$, Spending Score = 17.11
Cluster 4: Income = 26.30 k$, Spending Score = 20.91

Customers in each cluster:
Cluster
0    81
1    39
2    22
3    35
4    23
Name: count, dtype: int64

Cluster Analysis:
         Customer_Count  Average_Income  Average_Spending_Score
Cluster
0                    81           55.30                   49.52
1                    39           86.54                   82.13
2                    22           25.73                   79.36
3                    35           88.20                   17.11
4                    23           26.30                   20.91
```

In [ ]: